

Университет ИТМО
Факультет программной инженерии и компьютерной техники

Системное и прикладное программное обеспечение
Программно-информационные системы

Отчет
о производственной, технологической практике
по теме «Разработка модуля
контекстно-зависимого автодополнения
запросов на YQL»

Автор: Смирнов Виктор Игоревич, Р34131

Руководитель от университета ИТМО:

Маркина Татьяна Анатольевна, старший преподаватель

Руководитель от профильной организации:

Логинов Иван Павлович, генеральный директор
ООО «Специальные Средства Программирования»

Санкт-Петербург, 2025

Содержание

1	Список сокращений и условных обозначений	2
2	Характеристика организации	2
3	Введение	2
4	Сбор и анализ требований	3
4.1	Требования к пользовательскому интерфейсу	3
4.2	Функциональные требования	3
4.3	Технические требования	4
5	Реализация автодополнения ключевых слов	4
6	Проектирование интерфейса источника имен объектов БД	7
7	Реализация автодополнения имен объектов БД	9
8	Заключение	13
9	Список использованных источников	13
10	Приложение	13

1 Список сокращений и условных обозначений

БД — база данных

СУБД — система управления базами данных

YDB — распределённая отказоустойчивая Distributed SQL СУБД

SQL — декларативный язык программирования для получения, создания, модификации и управления данными в реляционной базе данных

YQL — универсальный декларативный язык запросов к YDB, диалект SQL

CLI — command-line interface

ANTLR — генератор парсеров по грамматике

GitHub — веб-сервис для хостинга IT-проектов и их совместной разработки

2 Характеристика организации

Основным видом деятельности общества с ограниченной ответственностью «Специальные Средства Программирования» являются научные исследования и разработки в области естественных и технических наук.

3 Введение

Целью практики по теме «Разработка модуля контекстно-зависимого автодополнения запросов на YQL» является повышение качества автодополнения запросов на YQL в YDB CLI.

Изначально автодополнение в данном клиентском приложении практически отсутствовало. Необходимо было сперва подготовить индивидуальное задание на практику, собрать и проанализировать требования к будущему решению, поставить задачу, выбрать подходящие технологии для реализации, разобраться в процессе разработки (сборка проекта, интеграция с другими модулями, наладить взаимодействие со смежными командами), а далее инкрементально решать поставленную задачу.

Вообще говоря, автодополнение кода является очень важным элементом любого приложения, предназначенного для ввода некоторого структурированного текста. Для реализации автодополнения необходимы знания формальных грамматик, синтаксического анализа текста, семантики языков программирования. Задача автодополнения запросов на SQL усложняется также тем, что требуется взаимодействовать с удаленным сервером для получения имен объектов БД, в то время как код классических языков программирования доступен локально и все данные можно получить из кодовой базы.

4 Сбор и анализ требований

Первый этап разработки включал в себя сбор и анализ требований к модулю автодополнения YQL и их представление в текстовом виде. Критерием выполнения являлось создание задач в сервисе GitHub в проектах YDB и YQL и описание требований в комментариях.

На данном этапе были налажена связь с представителями команды разработки YDB SDK, которые отвечают за YDB CLI. У них не было конкретных сформулированных требований, так что их пришлось выяснять в процессе переговоров.

4.1 Требования к пользовательскому интерфейсу

Были выделены следующие требования к пользовательскому интерфейсу:

- (UIR1) в интерактивном режиме YDB CLI должно осуществляться автодополнение запросов на YQL;
- (UIR2) по нажатию на клавишу TAB на экране должен быть отображен список кандидатов на автодополнение.

4.2 Функциональные требования

Были выделены следующие функциональные требования:

- (FR1) автодополнение должно исключать кандидатов, не подходящих по синтаксическому контексту;
- (FR2) автодополнение должно предлагать именованные объекты базы данных.

4.3 Технические требования

Были выделены следующие технические требования:

- (TR1) модуль автодополнения должен быть реализован на языке программирования C++;
- (TR2) сборка модуля автодополнения должна осуществляться при помощи системы Yatool;
- (TR3) автодополнение должно завершать работу не более чем за 0.5 секунд.

Приведенные выше требования были описаны в комментариях к задачам. Как только требования были согласованы, можно было продолжить работу в заданном направлении.

5 Реализация автодополнения ключевых слов

Данный этап включает в себя реализацию автодополнения ключевых слов в модуле автодополнения YQL и интеграция модуля в приложение YDB CLI. Критерием выполнения данного этапа является создание модуля автодополнения YQL, реализованный алгоритм возвращающий ключевые слова в качестве кандидатов на автодополнение, автодополнение ключевых слов в YDB CLI в интерактивном режиме.

В качестве основы автодополнения была выбрана библиотека antlr4-c3. Она принимает на вход текст и грамматику языка в формате ANTLR4, а возвращает список лексем, которые ожидает парсер в заданной позиции.

Конечно, необходимо было скрыть библиотеку за интерфейсом движка автодополнения.

```

struct TCompletionInput {
    TStringBuf Text;
    size_t CursorPosition = Text.length();
};

enum class ECandidateKind {
    Keyword,
    ...
};

struct TCandidate {
    ECandidateKind Kind;
    TString Content;
    ...
};

struct TCompletion {
    ...
    TVector<TCandidate> Candidates;
};

class ISqlCompletionEngine {
public:
    ...
    virtual TCompletion Complete(TCompletionInput input) = 0;
    ...
};

```

Листинг 1 — Интерфейс движка автодополнения

Для выдачи ключевых слов достаточно было удалить лексемы, соответствующие, например, числам и именам. Кроме того, antlr4-c3 выдает сразу последовательности лексем, если последовательность однозначна.

```

TLocalSyntaxContext::TKeywords SiftedKeywords(
    const TC3Candidates& candidates) {
    const auto& vocabulary = Grammar->GetVocabulary();
    const auto& keywordTokens = Grammar->GetKeywordTokens();

    TLocalSyntaxContext::TKeywords keywords;
    for (const auto& token : candidates.Tokens) {
        if (keywordTokens.contains(token.Number)) {
            auto display = Display(vocabulary, token.Number);
            auto& following = keywords[std::move(display)];
            for (auto next : token.Following) {
                following.emplace_back(Display(vocabulary, next));
            }
        }
    }
    return keywords;
}

```

Листинг 2 — Просеивание ключевых слов

Также необходимо было решить следующую проблему. Дело в том, что в SQL ключевые слова не являются полностью зарезервированными, из-за чего могут являться корректными именами, что отражено в грамматике и из-за чего antlr4-c3 возвращал почти все ключевые слова в каждой позиции, где ожидалось имя. Проблема была решена правильной настройкой antlr4-c3 – игнорированием некоторых правил вывода.

```

const TVector<TRuleId> KeywordRules = {
    RULE(Keyword),
    RULE(Keyword_expr_uncompat),
    RULE(Keyword_table_uncompat),
    RULE(Keyword_select_uncompat),
    RULE(Keyword_alter_uncompat),
    RULE(Keyword_in_uncompat),
    RULE(Keyword_window_uncompat),
    RULE(Keyword_hint_uncompat),
    RULE(Keyword_as_compat),
    RULE(Keyword_compat),
};

```

Листинг 3 — Правила вывода, подлежащие игнорированию

Далее модуль автодополнения был внедрен в YDB CLI. В качестве цвета для ключевых слов был выбран синий (в соответствии с темой Monaco). Сделать это было не очень сложно, ведь для реализации интерактивного решения в клиентском приложении используется библиотека Replxx, предоставляющая интерфейс для интеграции автодополнения и подсветки.

```

TCompletions Apply(
    const std::string& prefix, int& /* contextLen */) override {
    auto completion = Engine->Complete({
        .Text = prefix,
        .CursorPosition = prefix.length(),
    });

    replxx::Replxx::completions_t entries;
    for (auto& candidate : completion.Candidates) {
        const auto back = candidate.Content.back();
        if (
            !IsLeftPunct(back) && back != '<'
            || IsQuotation(back)) {
            candidate.Content += ' ';
        }

        entries.emplace_back(
            std::move(candidate.Content),
            ReplxxColorOf(candidate.Kind));
    }
    return entries;
}

```

Листинг 4 — Использование API Replxx

Весь код доступен для чтения в репозитории проекта YDB на GitHub.

6 Проектирование интерфейса источника имен объектов БД

Автодополнение ключевых слов уже заметно повысило пользовательский опыт разработки запросов на YQL в YDB CLI, но этого не достаточно для удобной работы. В любой удобной IDE реализованы подсказки также и имен различных сущностей, что очень ценят программисты, так как имена могут быть длинные, из-за чего их долго набирать, причем высок риск появления опечаток, а еще такие имена часто забываются.

Автодополнение имен является непростой задачей, так как требует также реализацию методов семантического анализа запроса с целью определения типа имени, требуемого в заданной позиции. Кроме того, необходим некоторый реестр имен.

Этот этап был посвящен проектированию интерфейса источника имен объектов БД для их получения в модуле автодополнения. Критерием выполнения была готовность спроектированного программного интерфейса

источника имен объектов БД для модуля автодополнения YQL, разработана in-memory реализация, служащая заглушкой.

К интерфейсу источника имен были выдвинуты следующие требования:

- Источник имен должен допускать реализацию через вызов удаленной процедуры;
- Источник имен должен поддерживать фильтрацию имен по заданным ограничениям;
- Источник имен должен поддерживать фильтрацию имен по содержимому;
- Источник имен должен ограничивать размер списка имен;
- Источник имен должен выдавать отсортированные по релевантности имена.


```

...
using TGenericName = std::variant<
    TKeyword,
    TPragmaName,
    TTypeName,
    ...>;

struct TNameRequest {
    TVector<TString> Keywords;
    struct {
        TMaybe<TPragmaName::TConstraints> Pragma;
        TMaybe<TTypeName::TConstraints> Type;
        ...
    } Constraints;
    TString Prefix = "";
    size_t Limit = 128;
    ...
}

struct TNameResponse {
    TVector<TGenericName> RankedNames;
};

class INameService {
public:
    ...
    virtual TFuture<TNameResponse> Lookup(
        TNameRequest request) = 0;
    ...
};
...

```

Листинг 5 — Интерфейс сервиса имен

Для модульного тестирования данного интерфейса также была разработана заглушка со списком имен, представленном прямо в коде.

Весь код доступен для чтения в репозитории проекта YDB на GitHub.

7 Реализация автодополнения имен объектов БД

В рамках данного этапа необходимо было подготовить пригодную для использования в YDB CLI реализацию сервиса имен, а также добавить элементы семантического анализа для определения типа имен и их последующей выдачи в списке кандидатов.

Более формально – реализация автодополнения имен объектов БД в модуле автодополнения. Критерий выполнения: разработан алгоритм возвра-

щающий имена объектов БД в качестве кандидатов на автодополнение, автодополнение имен наиболее важных типов объектов БД, установленных на этапе анализа требований, в YDB CLI в интерактивном режиме.

Представитель команды YQL очень помог мне с данной задачей, предоставив данные о частоте использования конструкций языка, а также списки имен функций, типов, прагм и прочего в файлах формата JSON.

Была реализована загрузка частот из JSON в следующую структуру.

```
struct TFrequencyData {
    THashMap<TString, size_t> Keywords;
    THashMap<TString, size_t> Pragmas;
    THashMap<TString, size_t> Types;
    THashMap<TString, size_t> Functions;
    THashMap<TString, size_t> Hints;
};
```

Листинг 6 — Структура данных с частотами имен

Далее с ее помощью реализован интерфейс стратегии ранжирования.

```
class IRanking {
public:
    ...
    virtual void CropToSortedPrefix(
        TVector<TGenericName>& names, size_t limit) = 0;
    ...
};
```

Листинг 7 — Интерфейс стратегии ранжирования

Ранжирование не только сортирует список, но и оставляет только первые k элементов. Это сделано, чтобы воспользоваться преимуществами алгоритма `std::partial_sort`.

```

...
void CropToSortedPrefix(TVector<TGenericName>& names, size_t limit)
override {
    limit = std::min(limit, names.size());
    ...
    ::PartialSort(
        std::begin(rows), std::begin(rows) + limit, std::end(rows),
        [this](const TRow& lhs, const TRow& rhs) {
            const size_t lhs_weight = ReversedWeight(lhs.Weight);
            const auto lhs_content = ContentView(lhs.Name);

            const size_t rhs_weight = ReversedWeight(rhs.Weight);
            const auto rhs_content = ContentView(rhs.Name);

            return std::tie(lhs_weight, lhs_content) <
                std::tie(rhs_weight, rhs_content);
        });
    ...
    names.crop(limit);
    ...
}
...

```

Листинг 8 — Реализация алгоритма ранжирования

Сами же имена были загружены из JSON в следующую структуру данных.

```

struct NameSet {
    TVector<TString> Pragmas;
    TVector<TString> Types;
    TVector<TString> Functions;
    THashMap<EStatementKind, TVector<TString>> Hints;
};

```

Листинг 9 — Структура данных с множеством имен

Далее необходимо было научиться распознавать типы имен в запросе. Реализовать это было не очень сложно, ведь, грубо говоря, antlr4-c3 можно сконфигурировать так, чтобы он возвращал parser call stack при нахождении имени.

```

bool IsLikelyPragmaStack(const TParserCallStack& stack);
bool IsLikelyTypeStack(const TParserCallStack& stack);
bool IsLikelyFunctionStack(const TParserCallStack& stack);
bool IsLikelyHintStack(const TParserCallStack& stack);
...
bool IsLikelyFunctionStack(const TParserCallStack& stack) {
    return EndsWith({
        RULE(Unary_casual_subexpr),
        RULE(Id_expr)
    }, stack) || EndsWith({
        RULE(Unary_casual_subexpr),
        RULE(Atom_expr),
        RULE(An_id_or_type)
    }, stack) || EndsWith({
        RULE(Atom_expr),
        RULE(Id_or_type)
    }, stack);
}
...

```

Листинг 10 — Распознавание имени функции по parser call stack

Достаточно интересным было распознавание имени пользовательской функции YQL. Дело в том, что оно состоит из пространства имен и непосредственно имени функции. Конечно, при набранном пространстве имен, движок должен предлагать только название функции.

```

TMaybe<TLocalSyntaxContext::TFunction> FunctionMatch(
    const NSQLTranslation::TParsedTokenList& tokens,
    const TC3Candidates& candidates) {
    if (!AnyOf(candidates.Rules, IsLikelyFunctionStack)) {
        return Nothing();
    }

    TLocalSyntaxContext::TFunction function;
    if (EndsWith(tokens, {"ID_PLAIN", "NAMESPACE"})) {
        function.Namespace = tokens[tokens.size() - 2].Content;
    } else if (EndsWith(tokens, {"ID_PLAIN", "NAMESPACE", ""})) {
        function.Namespace = tokens[tokens.size() - 3].Content;
    }
    return function;
}

```

Листинг 11 — Обработка имени функции

8 Заключение

В заключение хочу сказать, что данный проект еще не готов полностью. Далее необходимо будет поддержать автодополнение имен таблиц, колонок, индексов, директорий. Во первых, это требует взаимодействие с сервером СУБД. Во-вторых, это требует реализации более сложных алгоритмов семантического анализа. Я продолжу этим заниматься и после производственной практики, пока все не будет готово.

Кроме того, в рамках производственной практики были выполнены и другие задачи, связанные с подсветкой синтаксиса YQL. Они не отражены в отчете, так как не соответствуют индивидуальному заданию на практику.

В процессе работы были выявлены некоторые недостатки библиотеки antlr4-c3. Например, невозможность получение parser call stack для ключевых слов, а также отсутствие генерации более длинных последовательностей лексем, даже если продолжение не является однозначным.

Также я столкнулся с проблемой поддержки обратной совместимости библиотек. Дело в том, что изменения в исходный код YDB CLI требовалось вносить через репозиторий проекта YDB, а в YQL, где располагался модуль автодополнения, через репозиторий проекта YTsauros. Из-за этого было невозможным атомарное изменение исходных кодов и YDB CLI, и YQL. Этот пример иллюстрирует преимущества разработки в монорепоитории.

Также хочу отметить полезность периодических встреч с руководителем. Во-первых, это мотивировало делать прогресс по задачам, чтобы продуктивно проводить встречи. Во-вторых, после встреч повышалась мотивация что-то делать, когда делать что либо не очень хотелось. Мне показалось это очень хорошим приемом для совместной разработки.

9 Список использованных источников

- [1] A. V. Aho, M. S. Lam, R. Sethi, и J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2008.
- [2] T. Parr, *The Definitive ANTLR 4 Reference*, 2nd ed. Pragmatic Bookshelf, 2013.

10 Приложение

```
#include "sql_complete.h"
```

```
#include <yql/essentials/sql/v1/complete/text/word.h>
```

```
#include <yql/essentials/sql/v1/complete/name/static/name_service.h>
```

```
#include <yql/essentials/sql/v1/complete/syntax/local.h>
```

```

#include <yql/essentials/sql/v1/complete/syntax/format.h>

#include <util/generic/algorithm.h>
#include <util/charset/utf8.h>

namespace NSQLComplete {

    class TSqlCompletionEngine: public ISqlCompletionEngine {
    public:
        explicit TSqlCompletionEngine(
            TLexerSupplier lexer,
            INameService::TPtr names,
            ISqlCompletionEngine::TConfiguration configuration)
            : Configuration(std::move(configuration))
            , SyntaxAnalysis(MakeLocalSyntaxAnalysis(lexer))
            , Names(std::move(names))
        {
        }

        TCompletion Complete(TCompletionInput input) {
            if (
                input.CursorPosition < input.Text.length() &&
                IsUTF8ContinuationByte(input.Text.at(input.CursorPosition)) ||
                input.Text.length() < input.CursorPosition) {
                ythrow yexception()
                    << "invalid cursor position " <<
input.CursorPosition
                    << " for input size " << input.Text.size();
            }

            TLocalSyntaxContext context = SyntaxAnalysis-
>Analyze(input);

            TStringBuf prefix = input.Text.Head(input.CursorPosition);
            TCompletedToken completedToken =
GetCompletedToken(prefix);

            return {
                .CompletedToken = std::move(completedToken),
                .Candidates = GetCandidates(std::move(context),
completedToken),
            };
        }

    private:

```

```

TCompletedToken GetCompletedToken(TStringBuf prefix) {
    return {
        .Content = LastWord(prefix),
        .SourcePosition = LastWordIndex(prefix),
    };
}

TVector<TCandidate> GetCandidates(TLocalSyntaxContext context,
const TCompletedToken& prefix) {
    TNameRequest request = {
        .Prefix = TString(prefix.Content),
        .Limit = Configuration.Limit,
    };

    for (const auto& [first, _] : context.Keywords) {
        request.Keywords.emplace_back(first);
    }

    if (context.Pragma) {
        TPragmaName::TConstraints constraints;
        constraints.Namespace = context.Pragma->Namespace;
        request.Constraints.Pragma = std::move(constraints);
    }

    if (context.IsTypeName) {
        request.Constraints.Type = TTypeName::TConstraints();
    }

    if (context.Function) {
        TFunctionName::TConstraints constraints;
        constraints.Namespace = context.Function->Namespace;
        request.Constraints.Function = std::move(constraints);
    }

    if (context.Hint) {
        THintName::TConstraints constraints;
        constraints.Statement = context.Hint->StatementKind;
        request.Constraints.Hint = std::move(constraints);
    }

    if (request.IsEmpty()) {
        return {};
    }

    // User should prepare a robust INameService
    TNameResponse response = Names-

```

```

>Lookup(std::move(request)).ExtractValueSync();

    return Convert(std::move(response.RankedNames),
std::move(context.Keywords));
}

TVector<TCandidate> Convert(TVector<TGenericName> names,
TLocalSyntaxContext::TKeywords keywords) {
    TVector<TCandidate> candidates;
    for (auto& name : names) {
        candidates.emplace_back(std::visit([&](auto&& name) ->
TCandidate {
            using T = std::decay_t<decltype(name)>;
            if constexpr (std::is_base_of_v<TKeyword, T>) {
                TVector<TString>& seq = keywords[name.Content];
                seq.insert(std::begin(seq), name.Content);
                return {ECandidateKind::Keyword,
FormatKeywords(seq)};
            }
            if constexpr (std::is_base_of_v<TPragmaName, T>) {
                return {ECandidateKind::PragmaName,
std::move(name.Indentifier)};
            }
            if constexpr (std::is_base_of_v<TTypeName, T>) {
                return {ECandidateKind::TypeName,
std::move(name.Indentifier)};
            }
            if constexpr (std::is_base_of_v<TFunctionName, T>) {
                name.Indentifier += "(";
                return {ECandidateKind::FunctionName,
std::move(name.Indentifier)};
            }
            if constexpr (std::is_base_of_v<THintName, T>) {
                return {ECandidateKind::HintName,
std::move(name.Indentifier)};
            }
        }, std::move(name)));
    }
    return candidates;
}

TConfiguration Configuration;
ILocalSyntaxAnalysis::TPtr SyntaxAnalysis;
INameService::TPtr Names;
};

```



```

ISqlCompletionEngine::TPtr MakeSqlCompletionEngine(
    TLexerSupplier lexer,
    INameService::TPtr names,
    ISqlCompletionEngine::TConfiguration configuration) {
    return ISqlCompletionEngine::TPtr(
        new TSqlCompletionEngine(lexer, std::move(names),
std::move(configuration)));
}

} // namespace NSQLComplete

template <>
void Out<NSQLComplete::ECandidateKind>(IOOutputStream& out,
NSQLComplete::ECandidateKind kind) {
    switch (kind) {
        case NSQLComplete::ECandidateKind::Keyword:
            out << "Keyword";
            break;
        case NSQLComplete::ECandidateKind::PragmaName:
            out << "PragmaName";
            break;
        case NSQLComplete::ECandidateKind::TypeName:
            out << "TypeName";
            break;
        case NSQLComplete::ECandidateKind::FunctionName:
            out << "FunctionName";
            break;
        case NSQLComplete::ECandidateKind::HintName:
            out << "HintName";
            break;
    }
}

template <>
void Out<NSQLComplete::TCandidate>(IOOutputStream& out, const
NSQLComplete::TCandidate& candidate) {
    out << "{" << candidate.Kind << ", \"" << candidate.Content
<< "\"}";
}

```